No SQL

Agenda

- **Introduction to NoSQL**
- **Some Important Concepts**
- **Categories of NOSQL**
- **Characteristics of NOSQL**
- **MongoDB**

Introduction to NOSQL

INTRODUCTION

- Relational DBMSs offer many services in the form of queries and are structured in the form of schema.
- However these may not be appropriate for applications like:
  - Email services (Gmail) which have a large number of users, but require only a restricted set of services
  - Social Media (Facebook) with billions of users, each having thousands of posts for which a structured DBMS offers a restrictive model

NOSQL DATABASE SYSTEMS

- With the rise of technology and use of computers, the data set is increasing exponentially, and rapidly being used as well
- This means we need a special class of systems that can manage large amounts of data in organizations like Google, Amazon, Facebook, X
- For this, there is a high need of some kind of distributed database systems as the data now is not available/accessed from one place, but distributed across several storage systems

NOSQL DATABASE SYSTEMS

- The term '***NOSQL***' represents '***N****ot* ***O****nly* ***SQL***'
- Many applications need systems other than traditional relational SQL systems to augment their data management needs.
- This term generally represents systems that are based on architectures other than the relational database model.
- Most NOSQL Systems are ***distributed databases*** or ***distributed storage systems*** that focus on semi-structured data storage, high performance, availability, data replication and scalability

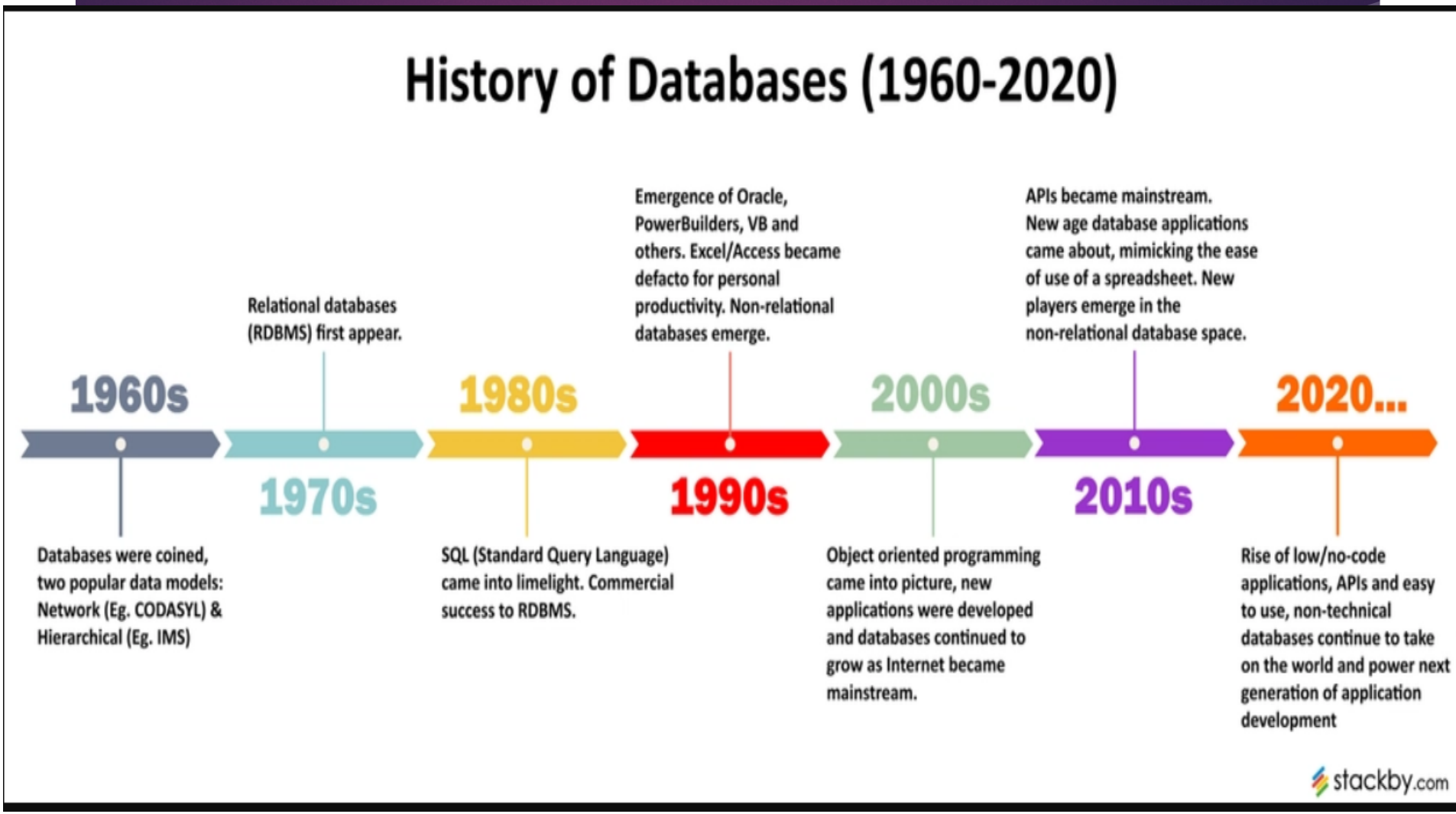
COMMERCIAL EXAMPLES

- Several organizations developed their own NOSQL models based on their data management requirements as follows:
  1. Google developed a proprietary NOSQL system called ***BigTable***. Google's innovation led to the category of NOSQL systems known as **column-based** or **wide column** stores
  2. Amazon developed a NOSQL system called ***DynamoDB***. This innovation led to the category known as **key-value data stores** or sometimes key-tuple or key-object data stores.
  3. Facebook developed a NOSQL system called ***Cassandra***, which uses concepts from **both** key-value stores and column-based systems.

COMMERCIAL EXAMPLES

- Several organizations developed their own NOSQL models based on their data management requirements as follows:
  4. Other software companies have made their own solutions and for users, like ***MongoDB*** and ***CouchDB***, developed their own system which are classified as **document-based** NOSQL system or document stores.
  5. Another category is the **graph-based** NOSQL system, or graph databases; these include ***Neo4J*** and ***GraphBase***, among others

Evolution of Database Systems



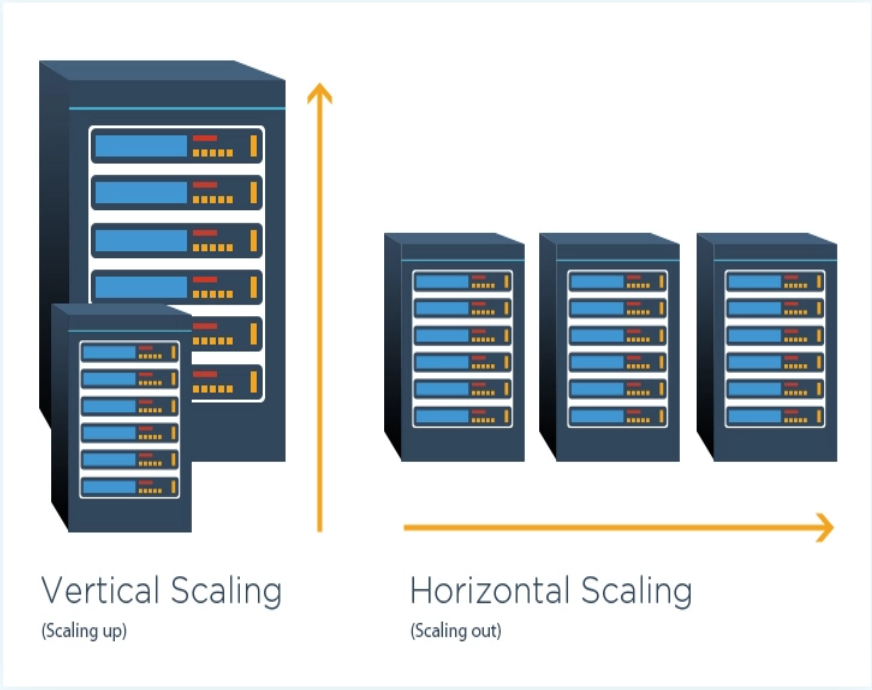
Some Important Concepts

SOME IMPORTANT CONCEPTS

- Some important concepts:
  - Scalability
  - Availability
  - Replication
  - Consistency

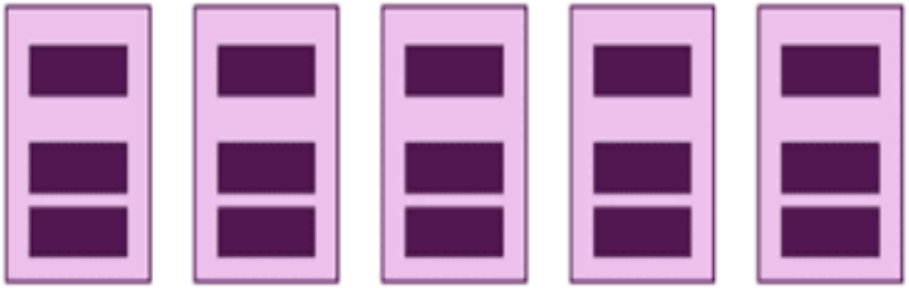
SCALABILITY

- As data grows exponentially, the systems storing that data should be capable to handle it efficiently
- Two kinds of scalability:
  - *Horizontal Scaling* refers to the addition of more nodes as the volume of data grows.
  - *Vertical Scaling* refers to expanding the computing and storage power of existing nodes.



SCALABILITY

- For growing data size in NOSQL systems, horizontal scalability is employed while the system is operational
- This can be achieved by techniques for distributing the existing data among new nodes without interrupting system operation.



Add More Servers (and merge Data into one Database)

AVAILABILITY, REPLICATION AND CONSISTENCY

- Many applications that use NOSQL systems require continuous system *availability* for its large number of users
- To accomplish this, data is *replicated* over nodes in a transparent manner, so that if one node fails, the data is still available on other nodes.
- *Replication* improves data availability and can also improve read performance.
- However, write performance becomes more cumbersome because an update must be applied to every copy of the replicated data items; this can slow down write performance if *consistency* is required.

REPLICATION MODELS

Two major models in use:

- ***Master-slave replication*** requires one copy to be the master copy; all write operations must be applied to the master copy and then propagated to the slave copies.
- For read, the master-slave paradigm can be configured in various ways, such as all reads required to be at the master copy.
- ***Master-master replication*** allows reads and writes at any of the replicas but may not guarantee that reads at nodes that store different copies see the same values.
- A reconciliation method to resolve conflicting write operations of the same data item at different nodes must be implemented.

SHARDING

- **In many NOSQL applications, files can have many millions of records, and these records can be accessed concurrently by thousands of users.**
- **It is not practical to store the entire file in one node.**
- ***Sharding* (also known as *horizontal partitioning*) of the file records is often employed in NOSQL systems.**
- **This serves to distribute the load of accessing the file records to multiple nodes.**

Categories of NOSQL

CATEGORIES OF NOSQL DATABASES

Four major Categories:

1. **Document based NOSQL Systems**
2. **NOSQL Key-Value Stores**
3. **Column based or Wide Column NOSQL Systems**
4. **Graph based NOSQL Systems**

DOCUMENT BASED NOSQL SYSTEMS

- **These systems store data in the form of documents using well-known formats, such as JSON (JavaScript Object Notation).**
- **Documents are accessible via their document id, but can also be accessed rapidly using other indexes.**
- ***MongoDB* and *CouchDB* are good example of document based NOSQL Systems.**

NOSQL KEY-VALUE STORES

- These systems have a simple data model based on fast access by the key to the value associated with the key.
- The value can be a record or an object or a document or even have a more complex data structure.
- Amazon's *DynamoDB* is an example of a key-value database.
- Another example is *Oracle's NOSQL Database*.

COLUMN BASED NOSQL SYSTEMS

- These systems partition a table by column into column families (a form of *vertical partitioning*)
- Each column family is stored in its own files. They also allow versioning of data values.
- Google's Cloud *BigTable* is an example of column-based NOSQL System

GRAPH BASED NOSQL SYSTEMS

- In these systems, data is represented as a graph, which is a collection of vertices (nodes) and edges.
- Both nodes and edges can be labeled to indicate the types of entities and relationships they represent,
- It is possible to store data associated with both individual nodes and individual edges.
- *Neo4j* is an example of a graph based NOSQL System. *Graphbase.AI* is another example.

Characteristics of NOSQL

CHARACTERISTICS OF NOSQL SYSTEMS

- **Dynamic schema:** Allow flexible shaping of data to meet new requirements without the need to migrate or change schemas.
- **Horizontal scalability:** They scale horizontally for adding more nodes into the existing ones and acquire enough storage for even bigger datasets and much higher traffic by distributing the load on multiple servers.
- **Document-based:** Data are presented in flexible, semi-structured formats like JSON/BSON (e.g., MongoDB).

CHARACTERISTICS OF NOSQL SYSTEMS

- **Key-value-based:** They possess a simple but fast access pattern (e.g., Redis) by storing data as pairs of keys and values.
- **Column-based:** Data are organized into columns instead of rows (e.g., CASSANDRA).
- **Distributed and high availability:** They are designed to be highly available and to automatically handle node failures and data replication across multiple nodes in a database cluster.

CHARACTERISTICS OF NOSQL SYSTEMS

- **Flexibility:** Allow developers to store and retrieve data in a flexible and dynamic manner, with support for multiple data types and changing data structures.
- **Performance:** Perfect for big data and real-time analytics and high volume applications.
- ***Less Powerful Query Languages*:** Applications that use NOSQL systems may not require SQL, because search (read) queries in these systems may locate single objects in single files.
- ***Versioning***: Some NOSQL systems provide storage of multiple versions of the data items, with timestamps of when the data version was created.

HIGH PERFORMANCE DATA ACCESS

- In many NOSQL applications, it is necessary to find individual records or objects (data items) from among the millions of data records or objects in a file.
- To achieve this, most systems use one of two techniques:
  - *Hashing* – The location of the data item is specified by a "key" and the record is determined by its hash value calculated using hash function, *h(k)*
  - *Range partitioning* – The location is determined by a range of key values. For example a location 'i' would hold objects whose key values lie in a range,
    $i_{min} < k < i_{max}$.
- The majority of accesses to an object will be using one of these techniques rather than by using complex query conditions.

MongoDB

DOCUMENT MODEL EXAMPLE

```
{
  "_id":
ObjectId("5ef2d4b45b7f11b6d7a"),
  "user_id": "Sherlock Holmes",
  "age": 40,
  "address":
          {
            “Country: “England”
            “City”: “London”,
            “Street”: “221B Baker St.”
          },
  “Hobbies”:[ violin, crime-solving ]
}
```

```
{
  "_id":
ObjectId("6ef8d4b32c9f12b6d4a"),
  "user_id": "John Watson",
  "age": 45,
  "address":
          {
            “Country: “England”
            “City”: “London”,
            “Street”: “221B Baker St.”
          },
  “Medical license”: “Active”
}
```

Source: MongoDB’s official slide decks

```
{
  "_id": ObjectId(
                "5f4f7fef2d4b45b7f11b6d7a"),
  "user_id": "Sean",
  "age": 29,
  "Status": "A"
}
```

The Document Model: Structure and Syntax

To the left is an example of a document representing a user details including user_id, age, and a status category.

Source: MongoDB's official slide decks

```
{
 "_id": ObjectId(
                "5f4f7fef2d4b45b7f11b6d7a"),
 "user_id": "Sean",
 "age": 29,
 "Status": "A"
}
```

The Document Model: Structure and Syntax

A document in MongoDB uses the JavaScript Object Notation (JSON) format.

This format uses curly brackets to mark the start and the end of the document.

Source: MongoDB's official slide decks

```
{
  "_id": ObjectId(
                "5f4f7fef2d4b45b7f11b6d7a"),
  "user_id": "Sean",
  "age": 29,
  "Status": "A"
}
```

The Document Model: Structure and Syntax

MongoDB refers to keys as fields.

The field-value pairs in a document are separated by colons (:)

Source: MongoDB's official slide decks

```
{
  "_id": ObjectId(
                "5f4f7fef2d4b45b7f11b6d7a"),
  "user_id": "Sean",
  "age": 29,
  "Status": "A"
}
```

The Document Model: Structure and Syntax

Each field must be enclosed within quotation marks. String values are often quoted as good practice.

Source: MongoDB's official slide decks

```
{
  "_id": ObjectId(
                "5f4f7fef2d4b45b7f11b6d7a"),
  "user_id": "Sean",
  "age": 29,
  "Status": "A"
}
```

The Document Model: Structure and Syntax

Each field-value pair is separated within the document by commas.

Source: MongoDB's official slide decks

COMPARING DATA MODELS

- Irrespective of the structure, each data model must have a proper way of defining the objects
- Once the objects have been created, we will need some kind of query language in order to access those objects
- MongoDB Query Language (MQL) is a query language that allows accessing Document Model in MongoDB
- The comparison is being done with Relational Database Systems and SQL in order to better understand the language. However, there is no direct relation between the two

DOCUMENT MODEL

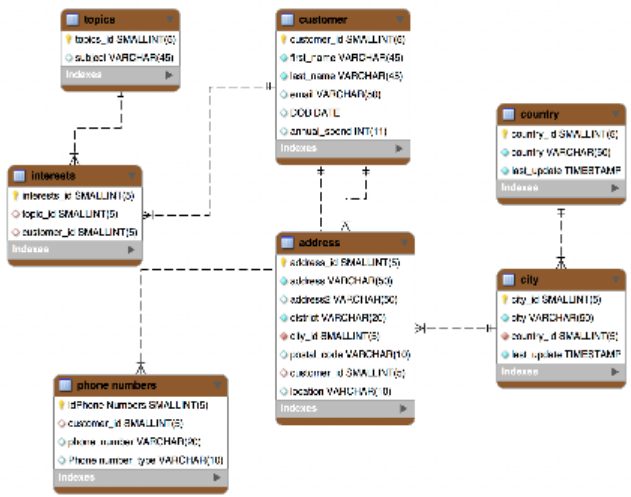
- In order to understand query structure in MongoDB, we must first have a good understanding of the ***Document Model*** and its constructs.
- A ***document*** is a way to organize and store data as a set of field-value pairs
- It represents information in a similar way as:
  - Dictionaries in Python
  - Maps in Java
  - JSON Objects in JavaScript

DOCUMENT MODEL

- In MongoDB, documents are actually stored as ***BSON*** (***B****inary* ***JSON***) in the underlying storage engine.
- Documents in MongoDB are presented via the drivers or via the MongoDB Shell as JSON
- JSON stands for **J**ava**S**cript **O**bject **N**otation. It is a format for storing and transporting data – mostly used when data is sent from a server to a web page
- JSON is "self-describing" and easy to understand, and MongoDB uses the same format to represent data items in the database

CONTRASTING DATA MODELS

- In ***Relational Model***, records are frequently split across records and tables

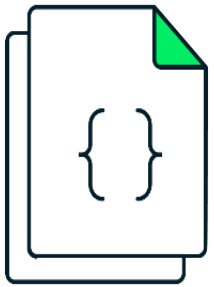


- In ***Document Model***, the data is typically defined within a single document

```
  "_id" : ObjectId("5ad88534e3632e1a35a58d00"),
  "name" : {
    "first" : "John",
    "last" : "Doe" },
  "address" : [
    { "location" : "work",
      "address" : {
        "street" : "16 Hatfields",
        "city" : "London",
        "postal_code" : "SE1 8DJ"},
      "geo" : { "type" : "Point", "coord" : [
          -0.109081, 51.5065752]}},
+   {...}
  ],
  "dob" : ISODate("1977-04-01T05:00:00Z"),
  "retirement_fund" : NumberDecimal("1292815.75")
}
```

DOCUMENT MODEL CONSTRUCTS

- The document model representation is based on three constructs:
  - ***Fields*** (also referred to as ***attributes***)
  - ***Sub-documents*** (also referred to as ***nested documents*** or ***objects***)
  - ***Arrays***



FIELDS/ATTRIBUTES

- In the Relational data model, a table would have a list of column names, which represent the table's ***attributes***, and each record (i.e. ***tuple***) would have a set of values in the same order as the column names.
- In MongoDB, using the document model, these column names correspond to the ***field*** names, whereas the relational database tuples correspond to the values of these field key-pairs.

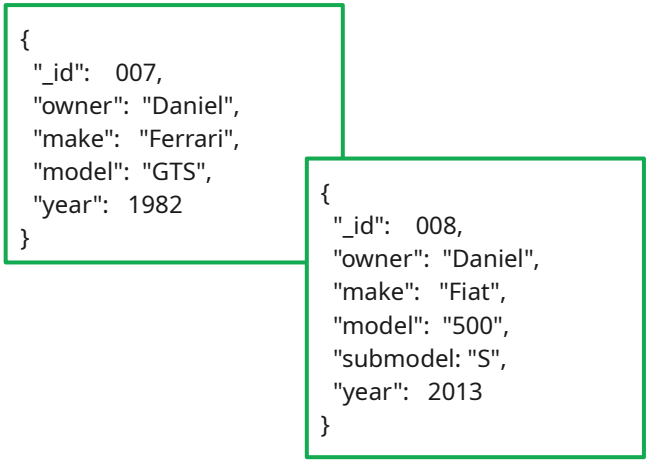
Cars				
_id	owner	make	model	year
007	Daniel	Ferrari	GTS	1982
Q08	Daniel	Fiat	500	2013

```
{
 "_id":   007,
 "owner":  "Daniel",
 "make":   "Ferrari",
 "model":  "GTS",
 "year":   1982
}
```

```
{
 "_id":   008,
 "owner":  "Daniel",
 "make":   "Fiat",
 "model":  "500",
 "submodel: "S",
 "year":   2013
}
```

FIELDS/ATTRIBUTES

- The main difference is that the attributes are appearing in each document, allowing for greater flexibility in having documents with different shapes; in other words, having different fields.
- Each document is self-describing without the need to refer to global metadata like a table definition



SUB-DOCUMENTS/OBJECTS

- An ***object***, or ***sub-document***, in the document model allows for information to be grouped together or embedded as a one-to-one relationship between what would be two tables from the tabular relational model.
- In the document data model, this would be within a document representing a car as an engine sub-document.

Cars		
_id	owner	make
007	Daniel	Ferrari
Q08	Daniel	Fiat

Engines			
_id	car_id	power	consumption
234808	007	660	10
Q08	008	120	45

```
{
  "_id": 007,
  "owner": "Daniel",
  "make": "Ferrari",
  "engine": {
        "power": 660hp,
        "consumption": 10mpg
  }
  ...
}
```

Tabular (Relational) Data Model

Header with column names
Row with values

Document Data Model

Each document explicitly list the names of the fields and the values.

ARRAYS

- A list of values that is enclosed in square brackets within a document represents a field whose value is an ***array***
- They represent the *denormalized pattern* of the relational database, which means that the “wheels” collection in the given table below, may not necessarily need a separate document, but can be embedded in the cars document as an array
- However, this does not mean they cannot be defined individually like in relational models

Cars		
_id	owner	make
007	Daniel	Ferrari
Q08	Daniel	Fiat

Wheels	
_id	car_id
234819	007
281928	007
392838	007
928038	007
950555	008

```
{
 _id: 007,
 "owner": "Daniel",
 "make": "Ferrari",
 wheels: [
          { "partNo": 234819 },
          { "partNo": 281928 },
          { "partNo": 392838 },
          { "partNo": 928038 }
 ],
 ...
}
```

MongoDB QUERY LANGUAGE

- The ***MongoDB Query Language*** or ***MQL***, was designed to have a simple syntax as well as being explicitly designed for querying documents.
- It only queries a single collection
- When you need to query multiple collections or perform more complex data processing, the MongoDB Aggregation Framework should be used.
- MQL is typically used for creating, reading, updating, or deletion (**CRUD**) operations.

MongoDB QUERY LANGUAGE

- MQL's query operators help enhance the MQL CRUD operations.
- MQL operators vary from comparison to logical to array, as well as specific purpose operators like bitwise or geospatial. These allow for complex operations to be performed on the documents.
- Some of these operators include:
  - Comparison
  - Logical
  - Element
  - Array
  - Evaluation
  - Bitwise
  - Comment
  - Geospatial
  - Projection, Update and Update Modifiers

MongoDB COMMANDS

- The command `show dbs;` is used to show all available databases.

```
> show dbs
sample_airbnb        52.82 MiB
sample_analytics      9.16 MiB
sample_geospatial     1.37 MiB
sample_guides        40.00 KiB
sample_mflix         50.33 MiB
sample_restaurants    7.04 MiB
sample_supplies       1.13 MiB
sample_training      52.30 MiB
sample_weatherdata    3.05 MiB
admin               280.00 KiB
local                 1.68 GiB
Atlas atlas-hggiwo-shard-0 [primary] test>
```

MongoDB COMMANDS

- In order to us… use <database>; command.

```
>_MONGOSH
> use sample_restaurants
< 'switched to db sample_restaurants'
```

- TIP: If t…
- To show… collec…

```
> show collections
< neighborhoods
  restaurants
Atlas atlas-hggiwo-shard-0 [primary] sample_restaurants>
```

MongoDB COMMANDS

- One of the ways to create a collection is to insert a document into a collection, as shown in the following example:

```
db.classical.insertOne(
    {"name": "Sultan" }
);
```

- Another command to create a collection is:

```
db.createCollection("classical");
```

```
> show collections
< neighborhoods
  restaurants
> db.classical.insertOne({"name" : "Sultan"})
< { acknowledged: true,
    insertedId: ObjectId("635ac8ff076fbc85d789c734") }
> show collections
< classical
  neighborhoods
  restaurants
Atlas atlas-hggiwo-shard-0 [primary] sample_restaurants>
```

MongoDB COMMANDS: `find()`

- The `find()` command is used to return documents satisfying given search criteria.
- To find all documents in a collection, use `db.<collection>.find({});`
- This command may return thousands of document (if in the collection).
- In order to return the first document in the collection, use `db.<collection>.findOne({});`

```
> db.restaurants.find({})
< { _id: ObjectId("5eb3d668b31de5d588f4292a"),
    address:
     { building: '2780',
       coord: [ -73.98241999999999, 40.579505 ],
       street: 'Stillwell Avenue',
       zipcode: '11224' },
    borough: 'Brooklyn',
    cuisine: 'American',
    grades:
     [ { date: 2014-06-10T00:00:00.000Z, grade: 'A', score: 5 },
       { date: 2013-06-05T00:00:00.000Z, grade: 'A', score: 7 },
       { date: 2012-04-13T00:00:00.000Z, grade: 'A', score: 12 },
       { date: 2011-10-12T00:00:00.000Z, grade: 'A', score: 12 } ],
    name: 'Riviera Caterer',
    restaurant_id: '40356018' }
  { _id: ObjectId("5eb3d668b31de5d588f4292b"),
```

MongoDB COMMANDS: `find()`

- We can find a document based on the position. For example, to find the third document in the collection, we can use the `skip` function:
  - `db.<collection>.find({}).skip(2);`
- This will display all the documents from the third one
- If we want to restrict it to just one document, we can use `limit` function:
  - `db.<collection>.find({}).skip(2).limit(1);`
- This will display only the third document
- Can use any number inside limit to restrict the number of documents to be found – any number beyond the size of document will be assumed as “until the end”

find() QUERIES: FINDING A DOCUMENT

- In order to find a document/documents with particular values for specific fields, the find command is used with {<field> : <value>, …} as a parameter.
- For example, in order to find all restaurants with Asian Cuisine, we formulate our query as:

```
db.restaurants.find({ “cuisine” : "Asian" });
```

- This may potentially return a large number of restaurants.
- To get a count of these restaurants (serving Asian cuisine) use

```
db.restaurants.find({ “cuisine” :
        "Asian" }).count();
```

find() QUERIES: FINDING A DOCUMENT

- For example, in order to find all restaurants with Asian Cuisine located in “Queens” borough, we formulate our query as:

```
db.restaurants.find({ “cuisine” : "Asian" ,
          “borough” : "Queens" })
```

- Can also use connectives like ***and*** and ***or***
- The above query with “and” can be rewritten as:

```
db.restaurants.find({$and:[{ “cuisine” :
  "Asian‟}, {“borough” : "Queens" }]})
```

`find()` QUERIES: FINDING A DOCUMENT

- Comparison operators can also be used as follows:
  - Equals → $eq
  - Not Equals → $ne
  - Less than → $lt
  - Less than or equal → $lte
  - Greater than → $gt
  - Greater than or equal → $gte
  - In → $in
  - Not In → $nin

QUERIES INVOLVING COMPARISONS

- Write a MongoDB query to find the restaurants which do not prepare American cuisine and located in “Queens” borough.
- `db.restaurants.find({"cuisine" : {$ne : "American "}, “borough" :{$eq: “Queens”}});`
- Using logical connectives
-
```
db.restaurants.find({ $and:
             [
               {"cuisine" : {$ne : "American "}},
               {“borough" :{$eq: “Queens”}}
                ]
              }
               );
```

find() QUERIES: FINDING A DOCUMENT

- Search documents if a specific field exists irrespective of the value in them
- Write a MongoDB query to find the restaurants which have borough fields defined for them.
- `db.restaurants.find({“borough" :{$exists: true}});`
- **OR**
- `db.restaurants.find({“borough" :{$exists: 1}});`

find() QUERIES

- You can use `find()` to return specific fields from the returned documents by using `db.<collection>.find(queries, projection, options)`
- Write a MongoDB query to display the fields restaurant_id, name, borough and cuisine for all the documents in the collection restaurants.
- `db.restaurants.find( {}, {" restaurant_id" : true, "name" : true, "borough" : true, "cuisine" : true})`

```
{ _id: ObjectId("5eb3d668b31de5d588f4293c"),
  borough: 'Brooklyn',
  cuisine: 'American',
  name: 'The Movable Feast',
  restaurant_id: '40361606' }
{ _id: ObjectId("5eb3d668b31de5d588f4293d"),
  borough: 'Manhattan',
  cuisine: 'Delicatessen',
  name: 'Bully\'S Deli',
  restaurant_id: '40361708' }
Type "it" for more
```

find() QUERIES

- “options” include commands like sort, limit, skip, and many more
- Depending on the environment being used, the options may need to be specified differently.
- In onecompiler that we are using in the course, options are written with a dot operator
- For example:
  - `db.restaurants.find( {}, {" restaurant_id" : true, "name" : true, "borough" : true, "cuisine" : true}).sort({"name": 1});`
  - This sorts the results in ascending, alphabetical order according to “name”

find() QUERIES: RESULTS

```
> db.restaurants.find({"cuisine" : "Asian"})
< { _id: ObjectId("5eb3d668b31de5d588f42b9f"),
    address:
     { building: '51',
       coord: [ -73.9787406, 40.7611474 ],
       street: 'West 52 Street',
       zipcode: '10019' },
    borough: 'Manhattan',
    cuisine: 'Asian',
    grades:
     [ { date: 2014-08-12T00:00:00.000Z, grade: 'A',
       { date: 2013-08-27T00:00:00.000Z, grade: 'A',
       { date: 2013-04-03T00:00:00.000Z, grade: 'B',
       { date: 2012-09-20T00:00:00.000Z, grade: 'A',
       { date: 2011-08-17T00:00:00.000Z, grade: 'A',
    name: 'China Grill',
```

```
> db.restaurants.find({"cuisine" : "Asian"}).count()
< 309
```

- In all, there are 309 restaurants in the collection serving “Asian cuisine”.
- Observe the “address” field in the document, which contains subfields building, coord, street, zipcode, …
- Write a query that finds all restaurants serving Asian cuisine on the Grand Street.

find() QUERIES: RESULTS

```
> db.restaurants.find({"cuisine" : "Asian", "address.street" : "Grand Street"})
< { _id: ObjectId("5eb3d668b31de5d588f43a8a"),
    address:
     { building: '245',
       coord: [ -73.994405, 40.718029 ],
       street: 'Grand Street',
       zipcode: '10002' },
    borough: 'Manhattan',
    cuisine: 'Asian',
    grades:
     [ { date: 2014-12-08T00:00:00.000Z, grade: 'A', score: 10 },
```

```
> db.restaurants.find({"cuisine" : "Asian", "address.street" : "Grand Street"}).count()
< 5
```

QUERIES INVOLVING COMPARISONS

- Find all restaurants which achieved a score of greater than 100.
  - ```
    db.restaurants.find({ “grades”:
                          { $elemMatch:
                             { “score”:
                                 { $gt:100 }
                             }
                          }
                        }
                       );
    ```
    ```

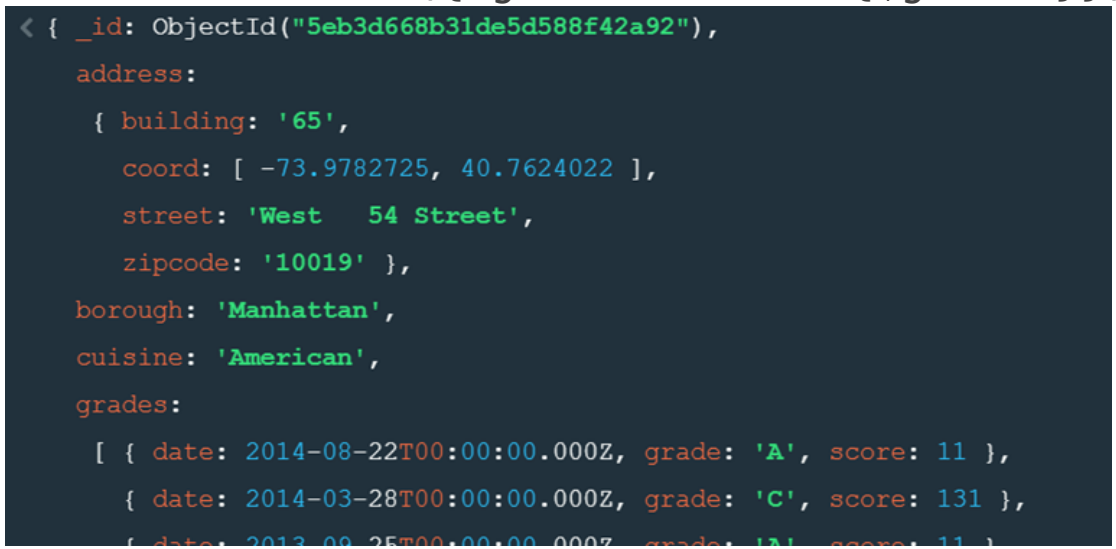
QUERIES INVOLVING COMPARISONS

- Find all restaurants which achieved a score of greater than 100.
  - `db.restaurants.find({“grades”:{$elemMatch:{“score”:{$gt:100}}}});`

```
> db.restaurants.find({grades : { $elemMatch:{"score":{$gt : 100}}}})
< { _id: ObjectId("5eb3d668b31de5d588f42a92"),
    address:
     { building: '65',
       coord: [ -73.9782725, 40.7624022 ],
       street: 'West   54 Street',
       zipcode: '10019' },
    borough: 'Manhattan',
    cuisine: 'American',
    grades:
     [ { date: 2014-08-22T00:00:00.000Z, grade: 'A', score: 11 },
       { date: 2014-03-28T00:00:00.000Z, grade: 'C', score: 131 },
```

QUERIES INVOLVING COMPARISONS

- Find all restaurants which achieved a score of greater than 100.
- Can also use dot operator to search for it as well
  - `db.restaurants.find({“grades.score”:{$gt:100}});`



QUERIES INVOLVING COMPARISONS

- Dot operator can be applied to any sub-document or an array of sub-documents
- However `$elemMatch` operator works for ONLY array of sub-documents
- For example, in the given example, $elemMatch will work for grades but not for address

QUERIES INVOLVING COMPARISONS

- Find all restaurants which achieved a score of greater than 100.
  - `db.restaurants.find({“grades”:{$elemMatch:{“score”:{$gt:100}}}});`
    OR
    `db.restaurants.find({“grades.score”:{$gt:100}});`
- Find all restaurants that are in building “65”
  - `db.restaurants.find({“address.building”:{$eq:“65”}});`
  - This works but if we apply $elemMatch, it won’t return any results, as it won’t find an “element” as a sub-document, i.e.
    - `db.restaurants.find({“address”:{$elemMatch:{“building”:{$eq:“65”}}}});`

QUERIES INVOLVING COMPARISONS

- Dot operator works on arrays of values as well using index number
- For example, in the given document, coord is an array of numbers, so we can use indices 0 and 1 with the dot operator to access them
- Example, find all restaurants whose first coordinate (i.e. at index 0) is 79.55
- `db.restaurants.find({“address.coord.0”:{$eq:79.55}});`

QUERIES INVOLVING COMPARISONS

- Write a MongoDB query to find the restaurants which do not prepare American cuisine and achieve a score more than 70 and located in the longitude less than -65.754168.

- ```
  db.restaurants.find({"cuisine" : {$ne : "American
  "}, "grades.score" :{$gt: 70}, "address.coord" :
  {$lt : -65.754168}});
  ```
  ```

QUERIES INVOLVING COMPARISONS

- Write a MongoDB query to find the restaurants which do not prepare American cuisine and achieve a score more than 70 and located in the longitude less than -65.754168.
- Can use logical connectives like and, or, not as well.
- For the given query, we can also use “and” as follows:
- ```
  db.restaurants.find({ $and:
                [
          {"cuisine" : {$ne : "American "}},
          {"grades.score" :{$gt: 70}},
          {"address.coord" : {$lt : -65.754168}}
                  ]
            }
                );
  ```
  ```

MongoDB

- Activity Sheet:
  - `Q1 – 3, 7, 10, 12, 16`

MongoDB INSERT

- In order to insert documents in a collection we use the following two variations:
- Observe that if the "_id" field is not specified, then it is created with a default ObjectId.
- Insert One document

```
db.restaurants.insertOne({name:"Asian"});
```

- Insert Many Documents

```
db.worker.insertMany([{_id: "W1",Ename: "John
Smith",ProjectID: "P1",Hours: 32.5} ,{_id:
"W2",Ename:  "Joyce English",ProjectID:
"P1",Hours: 20.0} ]);
```

MongoDB UPDATE

- To update a single document we use the command:

```
db.inventory.updateOne({ item: "paper" },
{$set: {  "size.uom": "cm", status: "P"}});
```

- If no item given in the first bracket, i.e. empty it automatically refers to the first document in the collection:

```
db.inventory.updateOne({},{$set:
{"size.uom": "cm", status: "P"}});
```

MongoDB UPDATE

- To update a many documents we use the command:

```
db.inventory.updateMany({ "qty": { $lt: 50 } },{$set: {  "size.uom": "in", status: "P"}});
```

- This updates all documents having “qty” field less than 50
- If no condition given in brackets, it applies to all documents by default

```
db.inventory.updateMany({},{$set: {"size.uom": "in",  status: "P"}});
```

MongoDB UPDATE OPERATORS

- Can apply the following operations:
  - Increment/Decrement → $inc
  - Renaming a field → $rename
  - Adding a new value to an existing set → $push
- For eg, update the inventory by increasing the qty of all items by 5
  - `db.inventory.updateMany({},{$inc: {“qty”:5}});`
- Can decrement with the same operator. Update the inventory by decreasing the qty of all items by 2
  - `db.inventory.updateMany({},{$inc: {“qty”:-2}});`
- Rename the field from “qty” to “quantity” for all documents
  - `db.inventory.updateMany({},{$rename: {“qty”: “quantity”}});`
- Adds a new status ‘S’ for all documents that have the status field
  - `db.inventory.updateMany({status:{$exists:1}},{$push: {“status”: “S”}});`

MongoDB DELETE

- The delete command removes documents from the collection.
- We can do bulk delete or delete a single document using the following commands:
  - `db.inventory.deleteOne({});`
  - `db.inventory.deleteMany({});`
  - `db.inventory.deleteMany({ status : "A" });`
  - `db.inventory.deleteOne({ status: "D" });`

This command deletes the first document that matches the given condition

MongoDB DELETE

- Activity Sheet:
  - Attempt Q31, 34, 36, 40, 43, 45

Goals of CC

1. **Maintain Data Consistency** – Ensure the database remains in a consistent state before and after transactions.
2. **Preserve Isolation** – One transaction should not see the intermediate results of another.
3. **Improve Throughput** – Allow maximum parallelism without conflicts.
4. **Prevent Anomalies** – Avoid issues like deadlocks, lost updates, and dirty reads.

THANK YOU